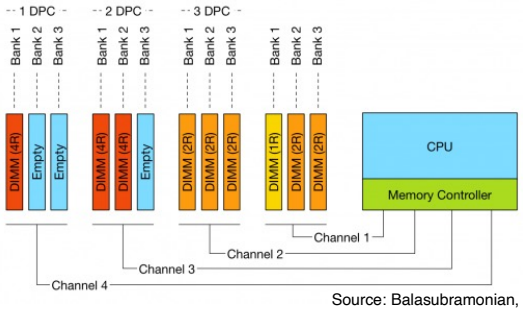
3. The SYSTEM Level

DIMM

Sources:
- Balasubramonian, Rajeev. Innovations in the Memory System, Springer, 2019.
- S. Rixner, W. Dally, U. Kapasi, P. Mattson, and J. Owens. Memory access scheduling. In Proc. of ISCA, 2000.

Source: https://frankdenneman.nl/2015/02/18/memory-tech-primer-memory-subsystem-organization/



Source: Balasubramonian, Rajeev. Innovations in the Memory System, Springer, 2019.

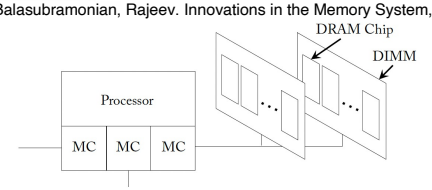


Figure 3.1: An example processor with three memory controllers and a basic DDR3 memory system with two DIMMs per channel.

Terminology

- *Memory channel*: Wires on a motherboard making up a data bus and an address/command bus.
  - The address/command bus is a unidirectional bus that carries a bits of address and c bits of command from the memory controller to memory chips.
  - The data bus is bidirectional and carries d bits of data from the memory chips to the memory controller for a read request, and vice versa for a write.
  - Assume that $a = 17$, $c = 5$, and $d = 64$.
- *DIMM*: The memory modules that plug into a memory channel.
  - Each DIMM is a printed circuit board (PCB) with memory chips on the front and back.
- *Rank*: A collection of DRAM chips on a DIMM that all work together to keep the 64-bit data bus busy in a cycle.
  - A single data wire on the memory channel is only connected to one DRAM chip pin on a rank.
  - An address/command wire on the memory channel is connected to every DRAM chip on every rank on the channel.

Source: Balasubramonian, Rajeev. Innovations in the Memory System, Springer, 2019.

Memory Pipeline

- Read request
  1. Request issued on address/command bus (~1 ns).
  2. Involved DRAM chips in a rank activate their circuits to pick out the required data (~35 ns).
  3. The requested data is sent back on the data bus (~5 ns).
- Three-stage *unbalanced* pipeline. *How to balance?*
- To fully utilize the data bus, multiple requests must be working on the second stage in parallel.

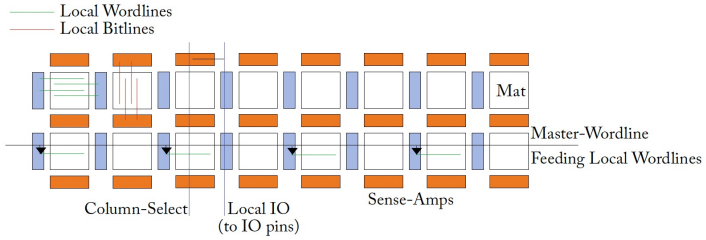


Figure 6.1: Two rows of mats in a bank are shown here. Each row is called a subarray. The sense-amplifiers are shown in orange, while the row decoders are shown in light blue. Other major interconnects are shown in green (local wordlines, implemented in poly), red (local bitlines, bottom metal layer), thick black (master wordline and bus feeding local IO, middle metal layer), and thick blue (column-select and local IO, top metal layer). Note that local wordlines and local bitlines are staggered.

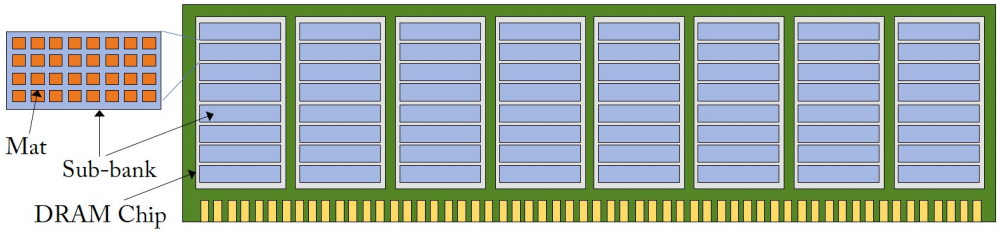


Figure 2.2: Example DIMM: 8 ×8 DRAM chips on one side form 1 rank and the back of the DIMM has another rank. A rank is partitioned into 8 banks. The portion of a bank in one chip can be called a sub-bank. The figure on the left zooms into one sub-bank, which is composed of 32 mats. Each row of mats is called a subarray.

- Sources of parallelism
  - Multiple *ranks* within a *channel* (possibly on different DIMMs).
  - Multiple *banks* within a rank.
  - Each bank is further decomposed into *sub-banks* (the part of a bank resident in a single DRAM chip; non-standard term). The banks typically share a bus that moves data between the banks and the I/O pins.
  - A bank is partitioned into multiple *subarrays* (a row of mats shown in Figure 6.1).
  - Each subarray is partitioned into a number of *mats*. A mat is typically 512 *rows* and 512 *columns* of DRAM cells.
- A data request involves several mats on several DRAM chips; activates a single wordline in each mat, with all DRAM cells in that row placing their data on bitlines.

Memory Access Cases

- *Page-hit* timing
  - Requested cache line is already in the row buffer.
  - Latency = time to ship the bits from the mat to the output pins (tCAS).
- *Page-empty* timing
  - Bit lines have been pre-charged, but row buffer is empty.
  - Latency = row activation time (tRCD) + transfer time.
- *Page-miss* timing
  - A different row currently occupies the row buffer.
  - Latency = precharge time (tRP) + row activation time + transfer time.

Case 1: For an **Idle Bank** the read latency is tRCD + tCAS

Case 2: For an **Active Bank** the read latency can vary

- → Minimum: tCAS for access to an active Page
- → Maximum: tRP + tRCD + tCAS when the Page must be closed to open another

If n% of accesses result in a hit to an open Page, then average read latency is:

$$n \times tCAS + (1-n) \times (tRP + tRCD + tCAS)$$

Then the break-even point for leaving the Page open or closing it would be:

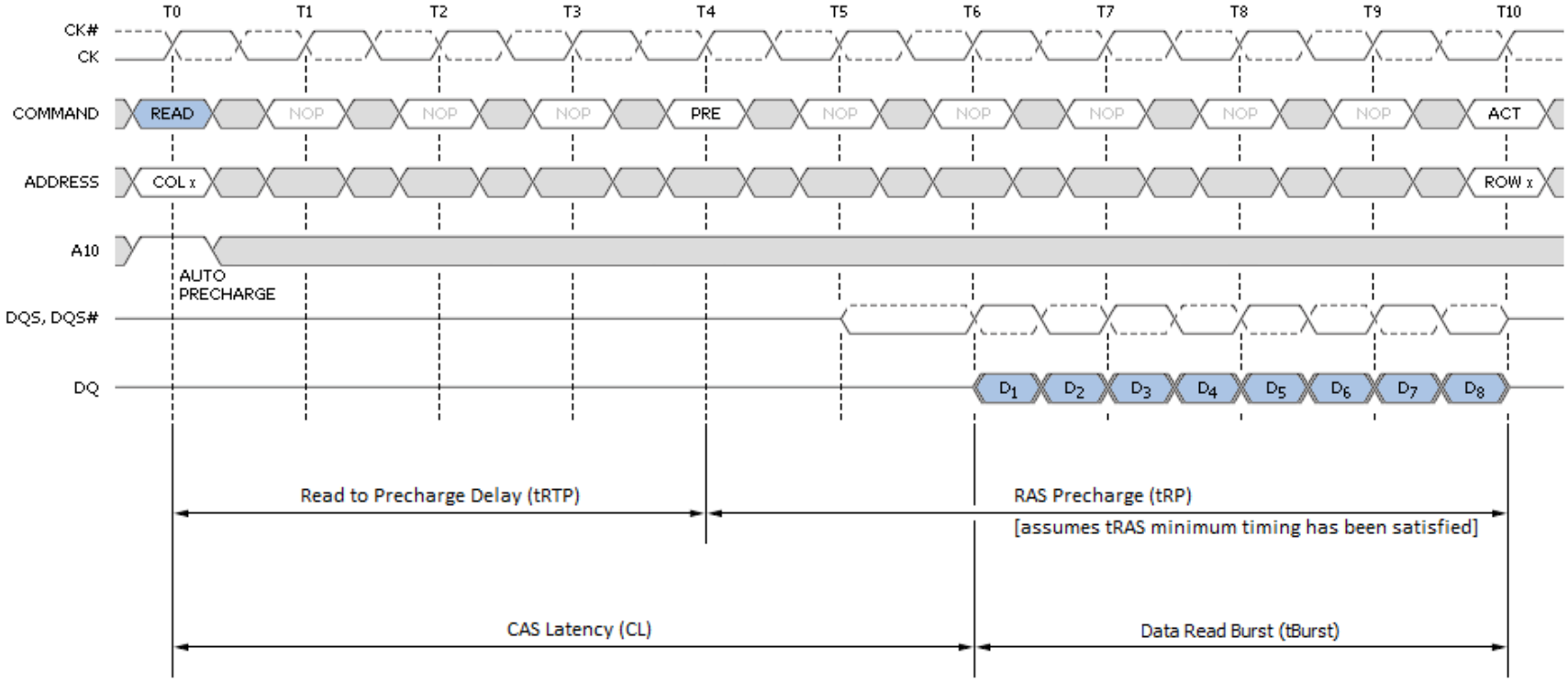
$$tRCD + tCAS = n \times tCAS + (1-n) \times (tRP + tRCD + tCAS)$$

$$n = \frac{tRP}{tRP + tRCD}$$

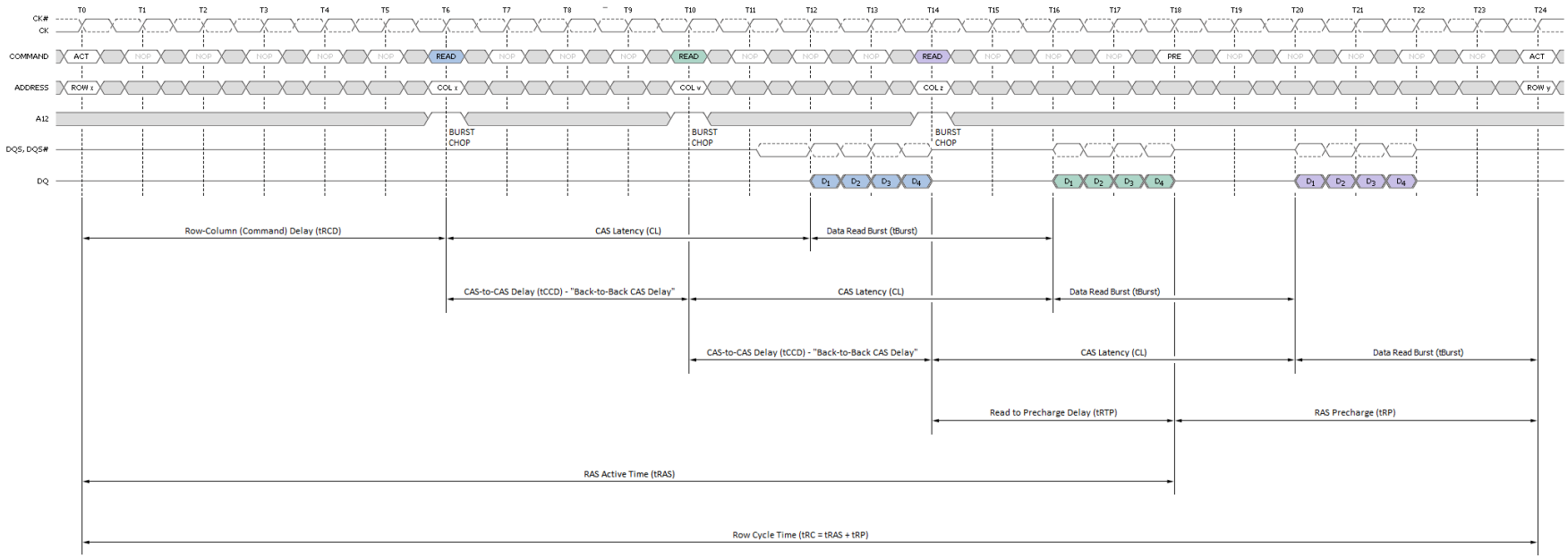
Source: https://www.anandtech.com/show/3851/everything-you-always-wanted-to-know-about-sdram-memory-but-were-afraid-to-ask/5

Source: https://images.anandtech.com/doci/3851/page_hit_auto_page_close.png

Page-Hit Access Timing (DDR3)

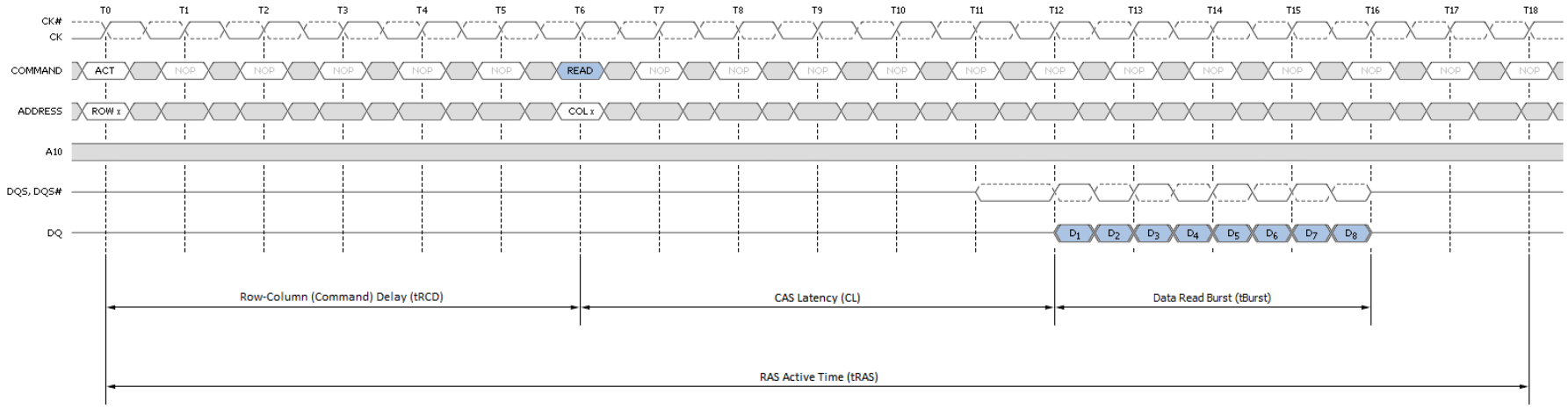


Page-Hit Access Timing (DDR3, triple burst)



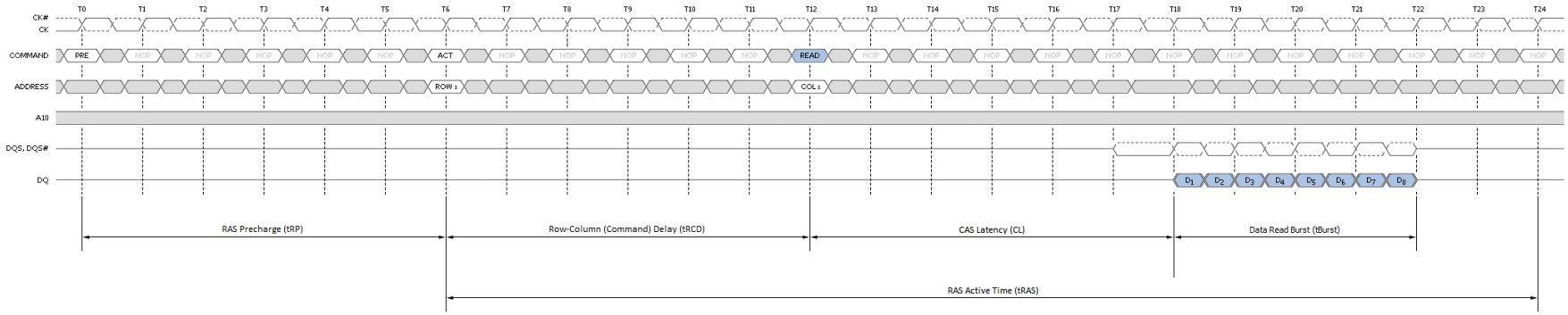
Source: https://images.anandtech.com/doci/3851/triple_burst_chop_with_page_close.png

Page-Empty Access Timing (DDR3)



Source: https://images.anandtech.com/doci/3851/page_empty_no_page_close.png

Page-Miss Access Timing (DDR3)

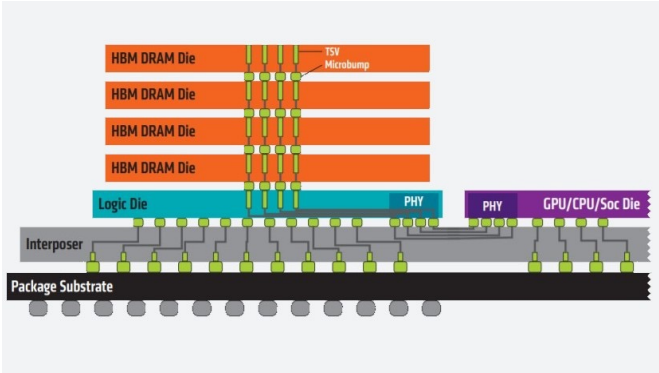


Source: https://images.anandtech.com/doci/3851/page_miss_no_page_close.png

HBM (High Bandwidth Memory)

Source: https://www.tomshardware.com/reviews/glossary-hbm-hbm2-high-bandwidth-memory-definition,5889.html

- 3D-stacked DRAM.
- Connected using through-silicon vias (TSVs) and microbumps and connect the GPU/CPU via the interposer, rather than on-chip.
- Meant to offer much higher bandwidth and lower power consumption.
- Very wide buses: two 128-bit channels per die initially.
  - 8-2023: Nvidia GH200 GPU, 144 GiB of HBM3e, 6144-bit bus, up to 5 TB/s.
  - 3-2024: Nvidia B200 GPU, 192 GiB of HBM3e, up to 8 TB/s.
- 🤔 If we built an L4$ with 1 GiB capacity using HBM, what block size would be needed so that the tag storage takes less than 1 MiB?



Type	Release	Max data rate speed per pin (Gbit/s)	Stack	per Stack	
				Max capacity (Number of dies × Die capacity = GByte)	Max data rate (GByte/s)
HBM 1	Oct 2013	1	8× 128 bit	4×1 = 4	128
HBM 2	Jan 2016	2.4		8×1 = 8	307
HBM 2E	Aug 2019	3.6		12×2 = 24	461
HBM 3	Jan 2022	6.4	16× 64 bit	12×2 = 24	819
HBM 3E	May 2023	9.8		16×3 = 48	1229
HBM 4	2026	6.4	32× 64 bit	16×4 = 64	1638

Source: https://en.wikipedia.org/wiki/High_Bandwidth_Memory

Memory Controller

The memory controller incorporates all the intelligence in the memory system.

- Can manage mapping of addresses to place data differently in memory.
  - To promote row buffer hits:

    Place consecutive cache lines in the same row.
    ```
    PAddr = {row, rank, bank,
    channel, column, blkoffset};
    ```
  - To promote parallelism:

    Place consecutive cache lines in different ranks and banks.
    ```
    PAddr = {row, column, rank, bank,
    channel, blkoffset};
    ```
- Must schedule memory operations, possibly reordering operations.
- Must issue timely refresh operations.

- DRAM refresh details
  - Retention time of 64 ms: Every cell must be refreshed within this interval.
  - Retention time halved to 32 ms at temperatures above 85 °C (extended temperature range).
  - Refresh of a memory rank is partitioned into 8,192 smaller refresh operations, one per 7.8 us (refresh interval tREFI).
  - Standard specifies that eight refresh intervals must be issued within 8 x tREFI.
  - Refresh operations are issued at the granularity of an entire rank.
    - Memory controller precharges all banks in the rank.
    - Then it issues a single auto-refresh command.
    - Low-level details of refresh are managed internally by the DRAM chips, using a row counter to keep track of the last refreshed row and using subarray-level parallelism.

Memory Scheduler: Basics

- On an LLC miss, a read request for a cache line is typically sent to the memory controller.
  - If the incoming cache line displaces a dirty line in the LLC, that dirty line is sent to the memory controller as a write request.
  - The memory controller has a *read queue* and a *write queue* to accommodate these incoming read/write requests.
- A canonical memory controller is free to reorder the requests in these read and write queues. Based on several heuristics, one of these requests is selected as highest priority and moved into *per-bank command queues*.
- Policy questions to populate command queues
  - Is a row buffer hit available in the read or write queues?
  - Are more row buffer hits likely or should a page be closed (should the bank be precharged)?
  - Should the next request be a read or a write?
  - Is now a good time to issue a refresh?
- The memory scheduler decomposes the request into multiple commands that are then placed in that bank's command queue.
  - For a request to an open row, a single column-read (CAS) command is enough to service a read request.
  - If the request causes a row conflict, then three commands are inserted into the command queue: precharge, activate (RAS), column-read (CAS).
- Once commands are placed in the command queue, they cannot be re-ordered.
- After satisfying timing constraints, the heads of each per-bank command queue are flagged as being ready or not.
- One of the ready commands is selected (e.g., using a round-robin scheme) and issued on the memory channel using a physical interface (aka PHY).

The First Ready, First Come First Served (FR-FCFS) Scheduler [Rixner *et al.* 2000]

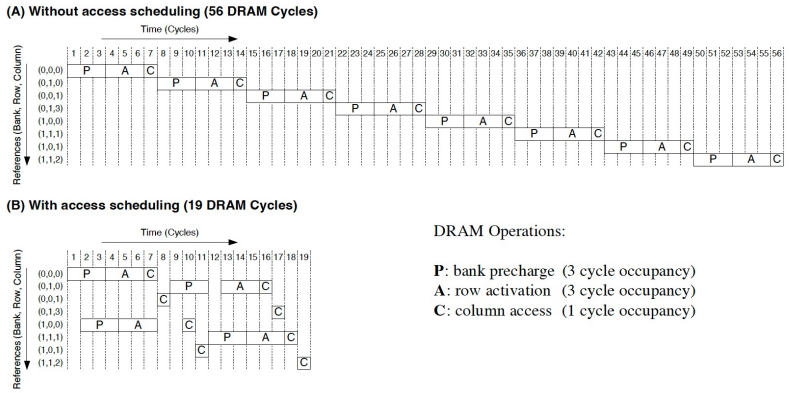


Figure 1. Time to complete a series of memory references without (A) and with (B) access reordering.

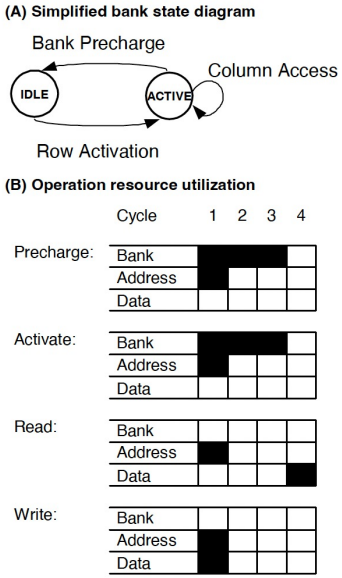


Figure 3. Simplified state diagram and resource utilization governing access to an internal DRAM bank.

Memory Access Scheduler Architecture

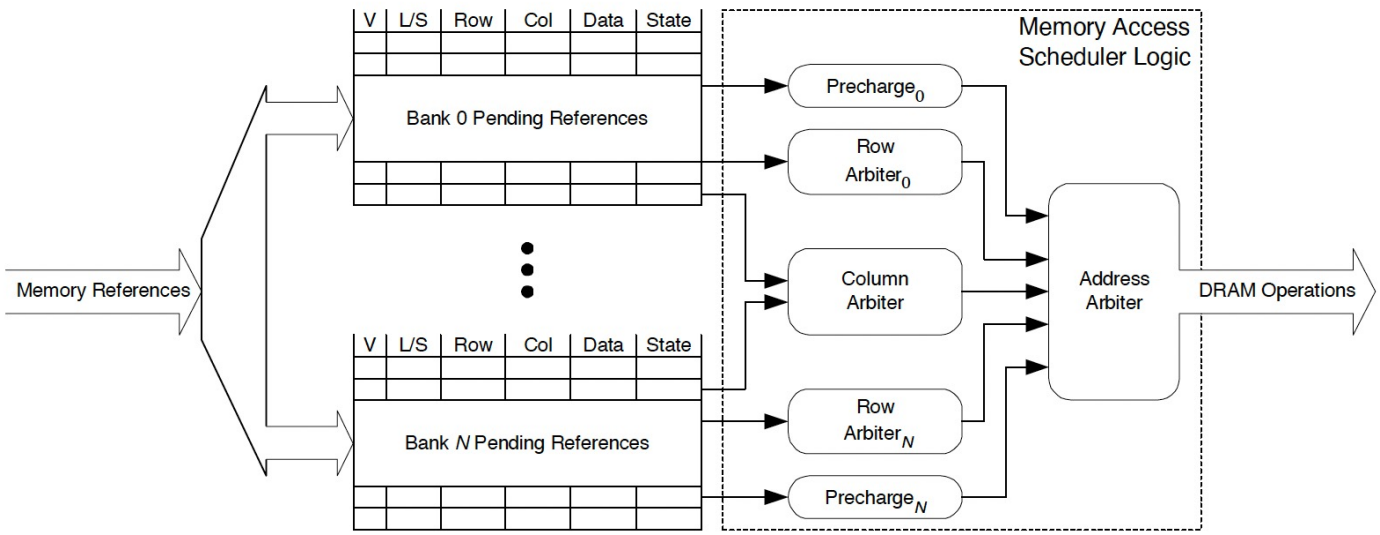


Figure 4. Memory access scheduler architecture.

Fields: valid (V), load/store (L/S), address (Row and Col), data, and whatever additional state is necessary for the scheduling algorithm.

Scheduling Policies

Table 1. Scheduling policies for the precharge managers, row arbiters, and column arbiter.

Policy	Arbiters	Description
in-order	precharge, row, and column	A DRAM operation will only be performed if it is required by the oldest pending reference. While used by almost all memory controllers today, this policy yields poor performance compared to policies that look ahead in the reference stream to better utilize DRAM resources.
priority	precharge, row, and column	The operation(s) required by the highest priority ready reference(s) are performed. Three possible priority schemes include: *ordered*, older references are given higher priority; *age-threshold*, references older than some threshold age gain increased priority; and *load-over-store*, load references are given higher priority. Age-threshold prevents starvation while allowing greater reordering flexibility than ordered. Load-over-store decreases load latency to minimize processor stalling on stream loads.
open	precharge	A bank is only precharged if there are pending references to other rows in the bank and there are no pending references to the active row. The open policy should be employed if there is significant row locality, making it likely that future references will target the same row as previous references did.
closed	precharge	A bank is precharged as soon as there are no more pending references to the active row. The closed policy should be employed if it is unlikely that future references will target the same row as the previous set of references.
most pending	row and column	The row or column access to the row with the most pending references is selected. This allows rows to be activated that will have the highest ratio of column to row accesses, while waiting for other rows to accumulate more pending references. By selecting the column access to the most demanded row, that bank will be freed up as soon as possible to allow other references to make progress. This policy can be augmented by one of the priority schemes described above to prevent starvation.
fewest pending	column	The fewest pending policy selects the column access to the row targeted by the fewest pending references. This minimizes the time that rows with little demand remain active, allowing references to other rows in that bank to make progress sooner. A weighted combination of the fewest pending and most pending policies could also be used to select a column access. This policy can also be augmented by one of the priority schemes described above to prevent starvation.